Università degli Studi di Napoli – Federico II

Progetto di macchine combinatorie

richiami e introduzione alla progettazione VHDL

ARCHITETTURA *dei* **SISTEMI DIGITALI**

Prof. A. De Benedictis

MACCHINE COMBINATORIE: richiami

Una macchina combinatoria è un circuito costituito da porte logiche, le cui uscite in ogni istante assumono valori che dipendono unicamente dal valore degli ingressi in quello stesso istante.

Le macchine combinatorie possono essere di vario tipo:

- Macchine per l'**istradamento di segnali** e la **connessione fra sistemi** (reti di priorità, encoder/decoder, multiplexer/demultiplexer)
- Macchine per la **conversione di codici** (reti di transcodifica)
- Macchine per la realizzazione di **funzioni aritmetiche e logiche** (sommatori, moltiplicatori, comparatori..)

Progetto di macchine combinatorie

- Il **metodo generale** per il progetto e la realizzazione di una macchina combinatoria prevede le seguenti fasi:
  1. **FORMALIZZAZIONE**: Dalla specifica semi-formale si passa ad una rappresentazione mediante **tabella di verità**
  2. **SINTESI**: Dalla tabella di verità si passa ad un' **espressione algebrica** (in <u>forma canonica</u> di tipo somma di prodotti o prodotto di somme)
  3. **OTTIMIZZAZIONE**: Dalla espressione in forma canonica si ottiene una espressione "migliore" e possibilmente ottimizzata secondo un certo parametro di qualità (tipicamente l'area occupata)
  4. **REALIZZAZIONE CIRCUITALE**: L'espressione ottimizzata viene tradotta in un circuito.

- Non sempre è possibile/conveniente procedere in questo modo:
  - A seconda dei requisiti da soddisfare e delle risorse a disposizione del progettista, potrebbe essere conveniente adottare un approccio modulare di **progettazione per composizione,** che si basa sull'utilizzo di macchine già esistenti
  - In base alla tecnologia realizzativa scelta, la fase di minimizzazione potrebbe essere inutile!

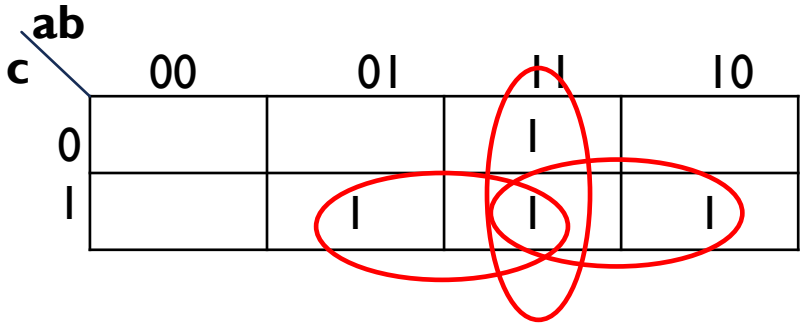
Esempio di progetto di una macchina combinatoria: funzione di maggioranza

Specifica: Si vuole progettare una macchina combinatoria che riceve in ingresso tre segnali binari **a**, **b** e **c**, e fornisce in uscita il valore che compare il maggior numero di volte nella stringa di ingresso

1-Formalizzazione:
tabella di verità

a	b	c	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

2-Sintesi:
f = !abc+a!bc+ab!c+abc (espressione in forma canonica SOP)



3-Ottimizzazione:
f =ab+ac+bc
(mappe di Karnaugh)

4-Realizzazione circuitale:
Nel caso in cui la tecnologia target fosse rappresentata dalle singole porte logiche elementari, assumendo che siano disponibili porte OR a tre ingressi, il circuito logico sarebbe perfettamente corrispondente alla espressione minima trovata

Progetto di un multiplexer 4:1 indirizzabile: metodo generale

Specifica: Si vuole progettare un multiplexer indirizzabile 4:1

Un multiplexer indirizzabile 4:1 è una <u>macchina notevole</u> con 4 ingressi binari **A0**,**A1**,**A2** e **A3**, $\log_2 4=2$ ingressi di selezione **a1** e **a0**, e una uscita binaria **U** su cui viene convogliato uno dei 4 ingressi in base al valore fornito come selezione

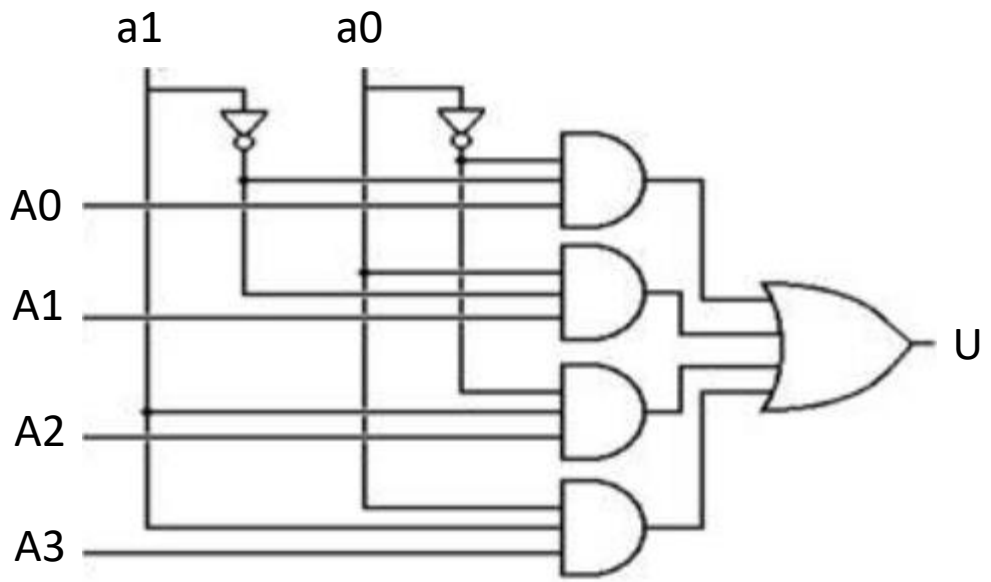
a1	**a0**	**U**
0	0	A0
0	1	A1
1	0	A2
1	1	A3

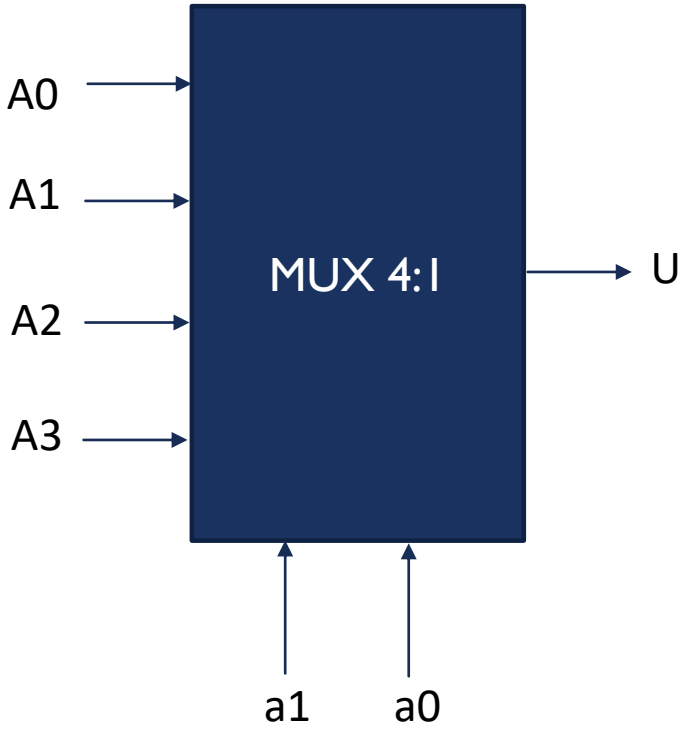
Tabella di verità sintetica

Espressione logica:

U=A0!a1!a0+A1!a1a0 + A2a1!a0 +A3a1a0

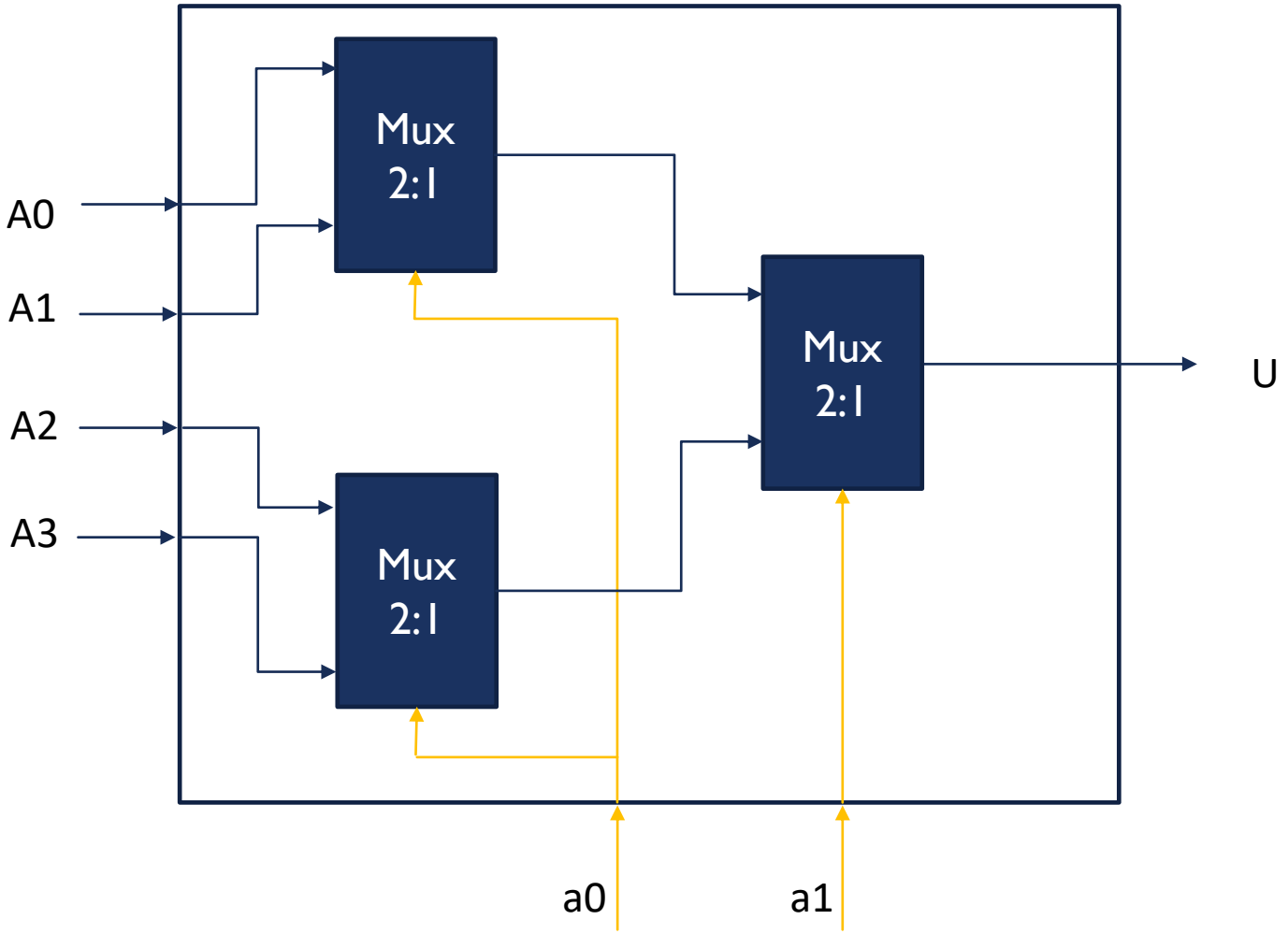
Circuito:





Progetto di un multiplexer indirizzabile 4:1: approccio modulare

Seguendo un **approccio modulare**, un mux 4:1 può essere visto come una composizione di macchine, ed in particolare è possibile utilizzare mux 2:1 opportunamente interconnessi:



Un simile approccio è notevolmente conveniente per semplificare il design nel caso in cui si abbia a disposizione una «libreria» di componenti già sviluppati

Il VHDL per la progettazione di sistemi elettronici digitali: introduzione

- Il VHDL è, insieme al Verilog, il linguaggio più usato per la progettazione di sistemi elettronici digitali
- Un progetto VHDL può essere trasformato direttamente in un formato sintetizzabile, utilizzabile per configurare un dispositivo FPGA o ASIC mediante un apposito tool detto sintetizzatore

- Rispetto agli approcci alla progettazione che abbiamo richiamato, il **VHDL consente di descrivere il comportamento di una macchina sia in termini dell'espressione logica realizzata che della struttura interna** intesa come interconnessione di componenti, utilizzando livelli di astrazione diversi.
- **In aggiunta**, il VHDL consente di descrivere come funziona una macchina **modellandone il comportamento ad alto livello** mediante costrutti tipici dei linguaggi di programmazione (for, if-the-else, ecc.)
  - NOTA: Il livello di astrazione utilizzato nel progetto influisce sulla sua sintetizzabilità, e livelli di astrazione differenti utilizzati nella descrizione di un circuito potrebbero dare luogo a implementazioni diverse nella tecnologia target

I costrutti fondamentali del VHDL: entity e architecture

La descrizione di una macchina in VHDL prevede di **definirne separatamente l'interfaccia e il comportamento**

Il costrutto **entity** consente di definire l'interfaccia di un componente (nome e porti di input/output)

```
entity example_code is port(
      port_1 : in std_logic;
      port_2 : out std_logic_vector(1 downto 0)
);
architecture example_code_arch of example_code is
  ...
begin

  ...

end example_code_arch;
```

Ogni porto ha un nome, una direzione, un tipo e una dimensione

Il costrutto **architecture** consente di definire il comportamento di un componente, specificato dopo la clausola **begin**
NOTA: la stessa entity può avere più implementazioni (definite con nomi diversi)

Esempio: descrizione VHDL MULTIPLEXER INDIRIZZABILE 2:1 - entity

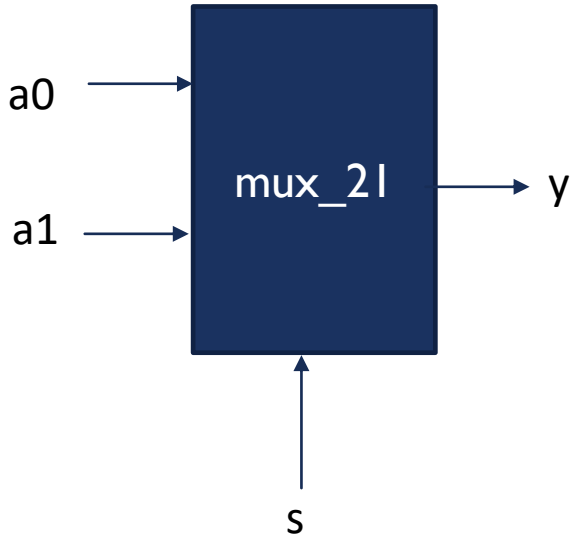
Per descrivere un multiplexer indirizzabile 2:1 in VHDL per prima cosa bisogna descrivere l'***entity,*** la scatola esterna

```
entity mux_2_1 is
    port(   a0  : in STD_LOGIC;
            a1  : in STD_LOGIC;
            s   : in STD_LOGIC;
            y   : out STD_LOGIC
    );
end mux_2_1;
```

In e **out** definiscono la direzione dei segnali, che sono rispettivamente di input e di output

std_logic è un **tipo** che definisce i valori che possono assumere i segnali digitali secondo lo standard IEEE 1164:

- 'U': uninitialized. This signal hasn't been set yet.
- 'X': unknown. Impossible to determine this value/result.
- '0': logic 0
- '1': logic 1
- 'Z': High Impedance
- 'W': Weak signal, can't tell if it should be 0 or 1.
- 'L': Weak signal that should probably go to 0
- 'H': Weak signal that should probably go to 1
- '-': Don't care.



Esempio: descrizione VHDL MULTIPLEXER INDIRIZZABILE 2:1 – architecture

Un primo approccio alla progettazione è quello di descrivere il mux 2:1 mediante l'espressione logica corrispondente, secondo il livello di astrazione **dataflow**

```
architecture dataflow of mux_2_1 is

    begin
        y <= ((a0 AND (NOT s)) OR (a1 AND s));

end dataflow;
```

Il costrutto principale usato nel livello dataflow è **l'assegnazione concorrente (<=)** in cui viene assegnato ad un segnale il valore risultante da un'espressione logica contenente 0 o più operatori logici

Esempio: descrizione VHDL MULTIPLEXER INDIRIZZABILE 2:1 – architecture dataflow

Un approccio simile utilizza il costrutto **when..else** (**conditional signal assignment**): il segnale assume un determinato valore a seconda della condizione che si verifica. E' possibile esprimere qualsiasi condizione booleana.

```
architecture alternative of mux_2_1 is

    begin
        y   <=  a0 when s = '0' else
                a1 when s = '1' else
                '-';

end alternative;
```

È necessario terminare il costrutto con un else incondizionato che tiene conto di tutti i casi non presi in considerazione dalle condizioni specificate

I casi **non sono mutuamente esclusivi** (come nel costrutto if dei linguaggi di programmazione): le condizioni vengono valutate **sequenzialmente** e la prima vera determina l'assegnazione.

Il costrutto è in genere *sintetizzabile* e dà luogo alla generazione di logica combinatoria, a meno che un segnale non venga assegnato a sè stesso, nel qual caso il tool di sintesi potrebbe generare dei latch

Esempio: descrizione VHDL MULTIPLEXER INDIRIZZABILE 2:1 – architecture dataflow

Alternativamente, è possibile utilizzare il costrutto **with..select (selected signal assignment)**

```
architecture alternative2 of mux_2_1 is

    begin
        with s select
            y   <=  a0 when '0',
                    a1 when '1',
                    '-' when others;

end alternative2;
```

il costrutto può essere applicato anche a segnali di più bit (array di segnali) ed è meno generale del costrutto when..else perché l'unica condizione valutata è quella di uguaglianza

È necessario enumerare tutte le possibili scelte (tutti i possibili valori che può assumere il segnale), altrimenti si può utilizzare «others»

I casi **non possono sovrapporsi** (è l'equivalente del costrutto **case** dei linguaggi di programmazione):

Esempio: descrizione VHDL MULTIPLEXER INDIRIZZABILE 2:1 – architecture behavioral

E' possibile adottare un livello di astrazione più alto (**behavioral**) che consiste nel descrivere semplicemente cosa fa la macchina utilizzando costrutti di alto livello

```
architecture behavioral of mux_2_1 is
    begin
        process (a0, a1, s)
            begin
            case s is
            when '0' => y <= a0;
            when '1' => y <= a1;
            when others => y <= '-';
            end case;
        end process;
end behavioral;
```

Il costrutto principale usato nel livello behavioral è il **process,** all'interno del quale è possibile definire degli statement sequenziali. Normalmente viene usato per descrivere circuiti sequenziali, ma può essere usato anche per definire logica puramente combinatoria mediante i costrutti **case** (equivalente al selected signal assignment select..with) e **if-then-else** (equivalente al conditional signal assignment when..else)

```
architecture behavioral1 of mux_2_1 is
    begin
        process (a0, a1, s)
            begin
                if (s='0') then
                    y <= a0;
                elsif (s='1') then
                    y <= a1;
                else
                    y <= '-';
                end if;
        end process;
end behavioral1;
```

Un process di solito ha una **sensitivity list** che contiene tutti i segnali in corrispondenza della cui variazione esso deve attivarsi.
Per assicurarsi che un process dia luogo a logica puramente combinatoria, tutti i segnali letti al suo interno devono essere inseriti nella sensitivity list

Progettazione modulare in VHDL: livello structural

```
architecture example_code_arch of example code is
      component instance1 is port(
            port_in : in std_logic;
            …
            port_out : out std_logic
      );

      signal sig_a : std_logic;
      signal sig_b : std_logic_vector(1 downto 0);
      signal sig_c : std_logic_vector(3 downto 0);
     ...

begin
      comp1 : instance1
      port map (
            port_in => sig_a;
            …
            port_out => sig_b;
      );

     ...
 end example_code_arch;
```

- All'interno del blocco *architecture* è possibile usare dei costrutti che permettono di descrivere il sistema come composizione di macchine secondo il livello di astrazione di tipo **structural**
- Tali costrutti consento di:
  - Dichiarare i componenti usati (in termini della loro interfaccia)
  - Istanziare e collegare fra loro i componenti specificando i loro ingressi/uscite effettivi (in termini di segnali)

Il costrutto **component** consente di dichiarare i componenti utilizzati da un design, mentre con **port map** è possibile istanziare il componente nella parte di descrizione funzionale (quella che inizia dopo **begin**)

Nella porzione riservata alle dichiarazioni è possibile dichiarare dei segnali (costrutto **signal**) interni al modulo che vengono utilizzati come segnali di appoggio o per l'interconnessione dei componenti interni

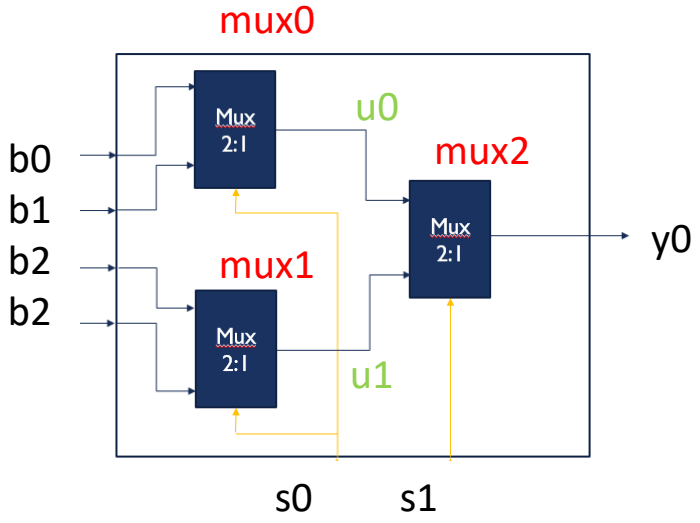
Esempio: descrizione VHDL MULTIPLEXER INDIRIZZABILE 4:1 – architecture structural

Seguendo l'approccio structural, possiamo descrivere un mux 4:1 come composizione di mux 2:1

```vhdl
entity mux_4_1 is
    port(   b0 : in STD_LOGIC;
            b1 : in STD_LOGIC;
            b2 : in STD_LOGIC;
            b3 : in STD_LOGIC;
            s0 : in STD_LOGIC;
            s1 : in STD_LOGIC;
            y0 : out STD_LOGIC
        );
end mux_4_1;

architecture structural of mux_4_1 is
    signal u0 : STD_LOGIC := '0';
    signal u1 : STD_LOGIC := '0';

    component mux_2_1
        port(   a0  : in STD_LOGIC;
                a1  : in STD_LOGIC;
                s   : in STD_LOGIC;
                y   : out STD_LOGIC
            );
    end component;

    begin
        mux0: mux_2_1
            Port map(   a0  => b0,
                        a1  => b1,
                        s   => s0,
                        y   => u0
                    );
        mux1: mux_2_1
            Port map(   a0  => b2,
                        a1  => b3,
                        s   => s0,
                        y   => u1
                    );
        mux2: mux_2_1
            Port map(   a0  => u0,
                        a1  => u1
                        s   => s1,
                        y   => y0
                    );
end structural;
```

I segnali dichiarati all'interno del blocco architecture servono per realizzare le interconnessioni

Usiamo il componente mux_2_1 progettato prima: di default viene utilizzata "the most recently analyzed architecture body"



Simulazione di un design: il testbench

Per verificare il funzionamento e la tempificazione di un progetto VHDL si usano i ***testbench***. Il testbench non è un oggetto che viene istanziato ma una design unit VHDL che:

- dichiara e **istanzia il blocco da testare**;
- applica un'opportuna sequenza di stimoli;
- verifica che le uscite siano quelle corrette, secondo le specifiche.

Il testbench non ha segnali di ingresso né di uscita perché **non può essere istanziato da nessun blocco** ma istanzia al suo interno i blocchi da testare.

Esempio di testbench per il mux 2:1

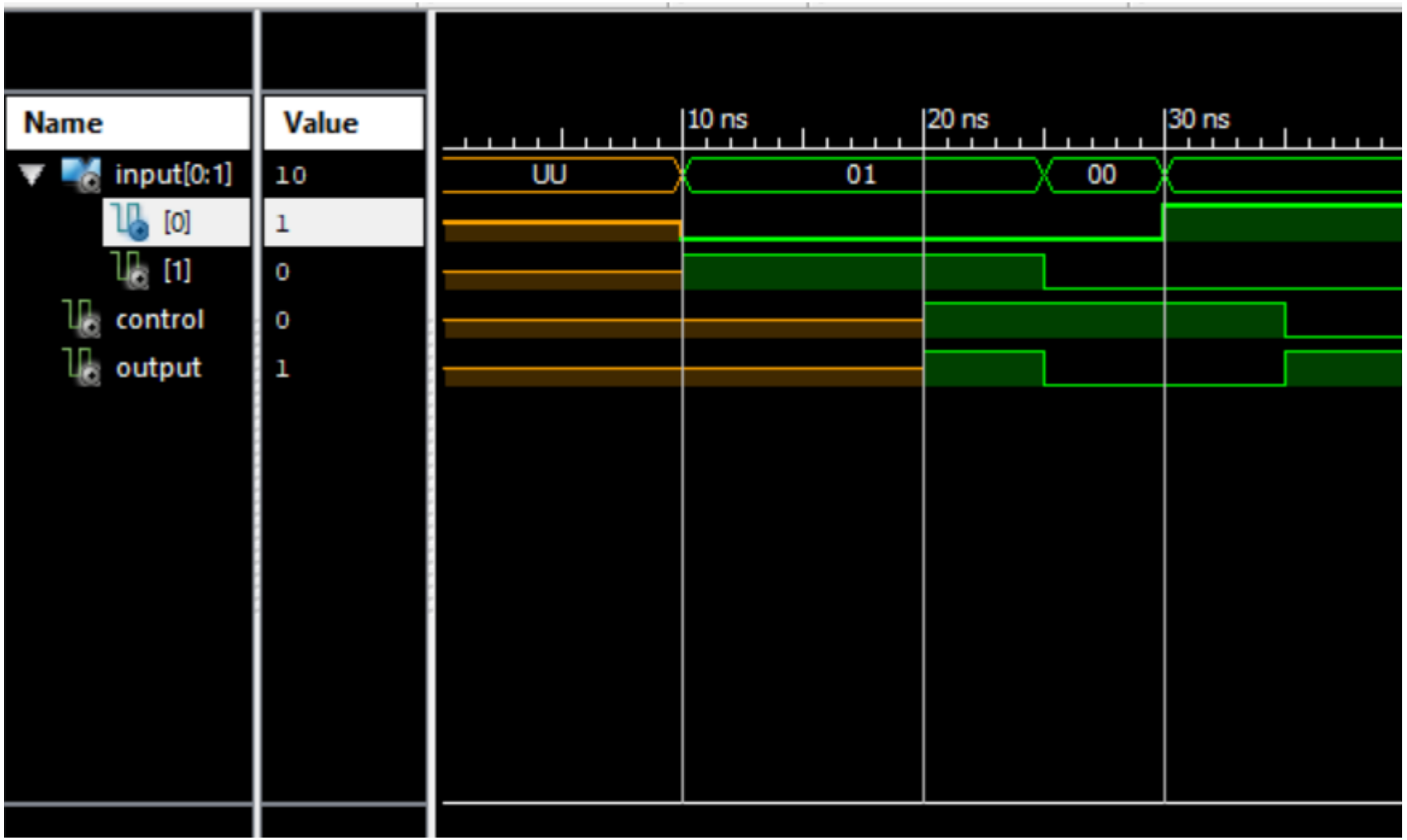
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.all;

entity mux_2_1_tb is
end mux_2_1_tb;

architecture mux_2_1 of mux_2_1_tb is

    component mux_2_1
        port(   a0 : in STD_LOGIC;
                a1 : in STD_LOGIC;
                s :  in STD_LOGIC;
                y : out STD_LOGIC
        );
    end component;

    signal input    : STD_LOGIC_VECTOR (0 to 1) := (others => 'U');
    signal control  : STD_LOGIC                 := 'U';
    signal output   : STD_LOGIC                 := 'U';

    begin
        -- specifico l'implementazione dell'entity che voglio simulare:
        utt : entity work.mux_2_1(dataflow) port map(
            a0 => input(0),
            a1 => input(1),
            s => control,
            y => output
        );
```

Di defaut, i file compilati in Xilinx ISE vengono inseriti nella libreria corrente chiamata «work»: per usare i componenti in essi definiti è necessario importare la libreria

In questo modo è possibile specificare il binding con l'implementazione da usare nel test

Le sequenze di dati in ingresso al modulo vengono fornite all'interno di un process.
Lo statement wait produce la sospensione del process per il tempo specificato

```
    stim_proc : process
    begin
    wait for 10 ns;
    input   <= "01"; -- visto che control='U' in uscita c'è ancora 'U'
    wait for 10 ns;
    control <= '1';   -- in uscita va a1='1'
    wait for 5 ns;
    input   <= "00"; -- in uscita va ancora a1, che stavolta è '0'
    wait for 5 ns;
    input   <= "10"; -- in uscita va ancora a1='0'
    wait for 5 ns;
    control <= '0';   -- in uscita va a0='1'
    assert output = '0'
    report "errore0"
    severity failure;
    wait;
    end process;

end;
```

La clausola assert consente di indicare l'output atteso: in caso di errore è possibile stampare un messaggio (report) e forzare l'arresto (severity failure)

Output della simulazione